A distributional analysis of QuickXsort for Mergesort

Jasper Ischebeck (Uppsala), joint work with
Florian Lesny and Ralph Neininger (both Frankfurt)

June 24, 2026

QuickMergeSort

- Sorting: Given a list of n elements $x_1, \dots, x_n \in \mathbb{R}$, return the sorted list
- One of the most basic and most used algorithms
- MergeSort is a fast sorting algorithm (only $n \log_2 n + \mathrm{O}(n)$ comparisons), but it needs $\Theta(n)$ additional storage
- QuickSort does not need much additional storage, but needs $n \ln n + \mathrm{O}(n)$ comparisons on average
- Idea: Can we combine MergeSort with QuickSort to not need so much additional storage?
- This is QuickMergeSort
- Aim: Show limiting distribution of the number of comparisons of QuickMergeSort in the uniform model and study this limit

QuickMergeSort

- Sorting: Given a list of n elements $x_1, \dots, x_n \in \mathbb{R}$, return the sorted list
- One of the most basic and most used algorithms
- MergeSort is a fast sorting algorithm (only $n \log_2 n + \mathrm{O}(n)$ comparisons), but it needs $\Theta(n)$ additional storage
- QuickSort does not need much additional storage, but needs $n \ln n + \mathrm{O}(n)$ comparisons on average
- Idea: Can we combine MergeSort with QuickSort to not need so much additional storage?
- This is QuickMergeSort
- Aim: Show limiting distribution of the number of comparisons of QuickMergeSort in the uniform model and study this limit

QuickMergeSort

- Sorting: Given a list of n elements $x_1, \dots, x_n \in \mathbb{R}$, return the sorted list
- One of the most basic and most used algorithms
- MergeSort is a fast sorting algorithm (only $n \log_2 n + \mathrm{O}(n)$ comparisons), but it needs $\Theta(n)$ additional storage
- QuickSort does not need much additional storage, but needs $n \ln n + \mathrm{O}(n)$ comparisons on average
- Idea: Can we combine MergeSort with QuickSort to not need so much additional storage?
- This is QuickMergeSort
- Aim: Show limiting distribution of the number of comparisons of QuickMergeSort in the uniform model and study this limit

QuickMergeSort

- Sorting: Given a list of n elements $x_1, \dots, x_n \in \mathbb{R}$, return the sorted list
- One of the most basic and most used algorithms
- MergeSort is a fast sorting algorithm (only $n \log_2 n + \mathrm{O}(n)$ comparisons), but it needs $\Theta(n)$ additional storage
- QuickSort does not need much additional storage, but needs $n \ln n + \mathrm{O}(n)$ comparisons on average
- Idea: Can we combine MergeSort with QuickSort to not need so much additional storage?
- This is QuickMergeSort
- Aim: Show limiting distribution of the number of comparisons of QuickMergeSort in the uniform model and study this limit

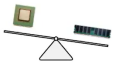
- Limit distribution is an open question posed by Wild last AofA
- Edelkamp, Weiß, and Wild (2020) derived, among other things, the asymptotic mean

Multiway Quicksort Summary

Multiway Quicksort

1. tricky **asymmetries** in *YBB Quicksort*
   ⇝ saves a few comparisons

2. evidence for growing weight of memory transfer costs



3. **Multiway Quicksort** reduces **memory transfer**

   - No other optimization of Quicksort achieves that

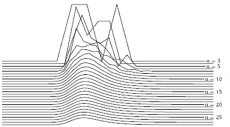
4. *AofA for Algorithm Science!*

Open Problem

- Automated/Convenient way to determine linear term of cost?

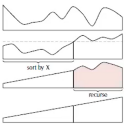
Much more to tell . . .

- Limiting **distributions** of costs



- **Sesquickselect** (cache-efficient selection)

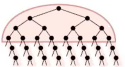
- ***QuickXsort*** inplace unstable mergesort



Open Problem

- limit distribution?

- Intriguing interplay of parameters



QuickXSort

- `QuickXSort` is a method that combines a sorting algorithm `XSort` with `QuickSort`:

$$\textit{SortingAlgorithms} \to \textit{SortingAlgorithms},$$
$$\texttt{XSort} \mapsto \texttt{QuickXSort}$$

- QuickMergeSort is the QuickXSort for XSort = MergeSort
- QuickMergeSort indeed has the speed of `MergeSort`,
- but no extra buffer is required

QuickXSort

- `QuickXSort` is a method that combines a sorting algorithm `XSort` with `QuickSort`:

$$\begin{aligned} \mathit{SortingAlgorithms} &\to \mathit{SortingAlgorithms}, \\ \texttt{XSort} &\mapsto \texttt{QuickXSort} \end{aligned}$$

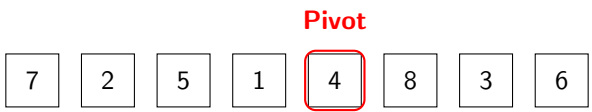
- QuickMergeSort is the QuickXSort for XSort = MergeSort
- QuickMergeSort indeed has the speed of `MergeSort`,
- but no extra buffer is required

QuickXSort

- Randomly (with some assumptions on distribution*) select an element as pivot
- Partition in elements smaller and larger than the pivot
- Use `XSort` on the bigger list if possible** with the other as buffer, permutating it
- Recurse: Use `QuickXSort` on the smaller list

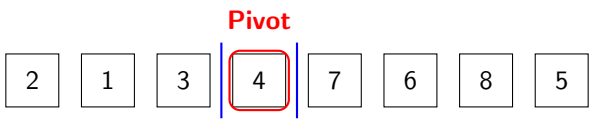
QuickXSort

- Randomly (with some assumptions on distribution*) select an element as pivot
- Partition in elements smaller and larger than the pivot
- Use `XSort` on the bigger list if possible** with the other as buffer, permutating it
- Recurse: Use `QuickXSort` on the smaller list



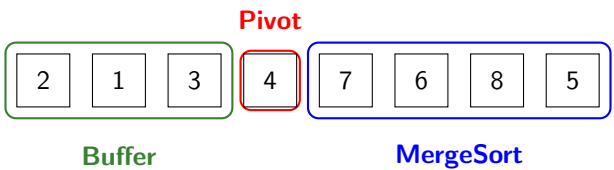
QuickXSort

- Randomly (with some assumptions on distribution*) select an element as pivot
- Partition in elements smaller and larger than the pivot
- Use `XSort` on the bigger list if possible** with the other as buffer, permutating it
- Recurse: Use `QuickXSort` on the smaller list



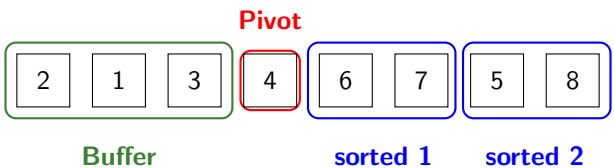
QuickXSort

- Randomly (with some assumptions on distribution*) select an element as pivot
- Partition in elements smaller and larger than the pivot
- Use `XSort` on the bigger list if possible** with the other as buffer, permutating it
- Recurse: Use `QuickXSort` on the smaller list



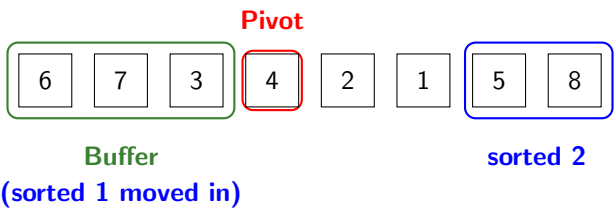
QuickXSort

- Randomly (with some assumptions on distribution*) select an element as pivot
- Partition in elements smaller and larger than the pivot
- Use `XSort` on the bigger list if possible** with the other as buffer, permutating it
- Recurse: Use `QuickXSort` on the smaller list



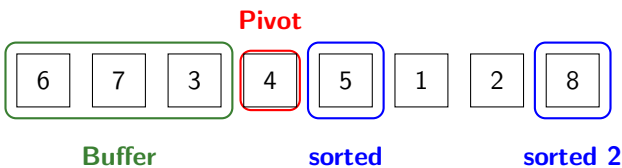
QuickXSort

- Randomly (with some assumptions on distribution*) select an element as pivot
- Partition in elements smaller and larger than the pivot
- Use `XSort` on the bigger list if possible** with the other as buffer, permutating it
- Recurse: Use `QuickXSort` on the smaller list



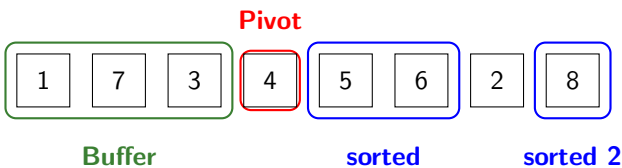
QuickXSort

- Randomly (with some assumptions on distribution*) select an element as pivot
- Partition in elements smaller and larger than the pivot
- Use `XSort` on the bigger list if possible** with the other as buffer, permutating it
- Recurse: Use `QuickXSort` on the smaller list



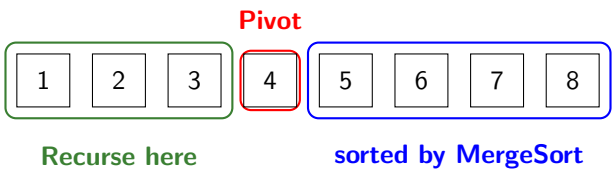
QuickXSort

- Randomly (with some assumptions on distribution*) select an element as pivot
- Partition in elements smaller and larger than the pivot
- Use `XSort` on the bigger list if possible** with the other as buffer, permutating it
- Recurse: Use `QuickXSort` on the smaller list



QuickXSort

- Randomly (with some assumptions on distribution*) select an element as pivot
- Partition in elements smaller and larger than the pivot
- Use `XSort` on the bigger list if possible** with the other as buffer, permutating it
- Recurse: Use `QuickXSort` on the smaller list



Assumptions on Pivots and `XSort`

Pivot Selection

Assume:

- elements $x_1, \ldots, x_n$ are distinct,
- the order of elements is uniform in S_n,
- pivot selection takes $o(n)$ time,
- relative pivot rank converges:

$$\frac{\#\{\text{elements left of pivot}\}}{n} \xrightarrow{d} Z'. \tag{P'}$$

Pivot Selection

Assume:

- elements $x_1, \ldots, x_n$ are distinct,
- the order of elements is uniform in S_n,
- pivot selection takes $o(n)$ time,
- relative pivot rank converges:

$$\frac{\#\{\text{elements left of pivot}\}}{n} \xrightarrow{d} Z'. \tag{P'}$$

Examples

- **random pivot**, first element or last element:

$$Z' \sim \mathrm{Unif}(0,1),$$

- **median-of-**$(2h-1)$: Sample $2h-1$ elements and take the median

$$Z' \sim \mathrm{Beta}(h,h).$$

- $\mathrm{Beta}(h,h)$ is more concentrated around $\frac{1}{2}$ for higher h

Examples

- **random pivot**, first element or last element:

$$Z' \sim \mathrm{Unif}(0,1),$$

- **median-of-**$(2h-1)$: Sample $2h-1$ elements and take the median

$$Z' \sim \mathrm{Beta}(h,h).$$

- $\mathrm{Beta}(h,h)$ is more concentrated around $\frac{1}{2}$ for higher h

XSort

- We assume that `XSort` needs a buffer of size $\sim \alpha n$ for $\alpha \leq 1$ and that it only permutes this buffer
- In order to do asymptotic analysis on `QuickXSort`, we need asymptotic analysis on `XSort`.
- Let M_n be the number of comparisons `XSort` needs to sort n elements in a uniformly random order.
- We assume

$$\begin{aligned} \mathbb{E}[M_n] &= n \log_2 n + \Pi(\log_2 n) n + o(n) \\ \mathrm{Var}(M_n) &= \mathrm{O}(n). \end{aligned} \tag{X}$$

  for some 1-periodic, continuous function Π.
- Fortunately, this has already been shown for some sorting algorithms

XSort

- We assume that `XSort` needs a buffer of size $\sim \alpha n$ for $\alpha \leq 1$ and that it only permutes this buffer
- In order to do asymptotic analysis on `QuickXSort`, we need asymptotic analysis on `XSort`.
- Let M_n be the number of comparisons `XSort` needs to sort n elements in a uniformly random order.
- We assume

$$\begin{aligned} \mathbb{E}[M_n] &= n \log_2 n + \Pi(\log_2 n) n + o(n) \\ \mathrm{Var}(M_n) &= \mathrm{O}(n). \end{aligned} \tag{X}$$

  for some 1-periodic, continuous function Π.
- Fortunately, this has already been shown for some sorting algorithms

$$\begin{aligned}\mathbb{E}[M_n] &= n\log_2 n + \Pi(\log_2 n)n + o(n)\\ \mathrm{Var}(M_n) &= \mathrm{O}(n).\end{aligned} \tag{X}$$

- (X) is true for Top-Down MergeSort, Hwang (1998), with

$$\Pi^{td}(x) = \frac{1}{2} - \frac{1}{\log(2)} + \frac{\Psi'(0)}{\log(2)} + \frac{1}{\log(2)} \sum_{k\in\mathbb{Z}\setminus\{0\}} \frac{1+\Psi(\chi_k)}{\chi_k\,(\chi_k+1)} e^{2k\pi i x}$$

with $\chi_k = \frac{2k\pi i}{\log(2)}$ and

$$\Psi(u) = \sum_{m\geq 1} \frac{2}{(m+1)(m+2)} \left(-\frac{1}{(2m)^u} + \frac{1}{(2m+1)^u} \right)$$

- and for QueueMergeSort (W.-M. Chen, Hwang, and G.-H. Chen 1999) with

$$\Pi^{q}(x) = 1 - \{x-\beta\} - 2^{1-\{x-\beta\}}$$

for $\beta = \sum_{j>0} \frac{1}{2^j+1}$ and $\{x\} := x - \lfloor x \rfloor$.

$$\begin{aligned}\mathbb{E}[M_n] &= n\log_2 n + \Pi(\log_2 n)n + o(n)\\ \mathrm{Var}(M_n) &= \mathrm{O}(n).\end{aligned} \tag{X}$$

- (X) is true for Top-Down MergeSort, Hwang (1998), with

$$\Pi^{td}(x) = \frac{1}{2} - \frac{1}{\log(2)} + \frac{\Psi'(0)}{\log(2)} + \frac{1}{\log(2)} \sum_{k\in\mathbb{Z}\setminus\{0\}} \frac{1+\Psi(\chi_k)}{\chi_k\,(\chi_k+1)} e^{2k\pi i x}$$

  with $\chi_k = \frac{2k\pi i}{\log(2)}$ and

$$\Psi(u) = \sum_{m\geq 1} \frac{2}{(m+1)(m+2)} \left(-\frac{1}{(2m)^u} + \frac{1}{(2m+1)^u} \right)$$

- and for QueueMergeSort (W.-M. Chen, Hwang, and G.-H. Chen 1999) with

$$\Pi^{q}(x) = 1 - \{x-\beta\} - 2^{1-\{x-\beta\}}$$

  for $\beta = \sum_{j>0} \frac{1}{2^j+1}$ and $\{x\} := x - \lfloor x \rfloor$.

Other XSort

- For the third common MergeSort, Bottom-Up MergeSort, (X), specifically $\mathrm{Var}(M_n) = \mathrm{O}(n)$ is not true
  - For $n = 2^m + 1$, the last element is inserted into a sorted 2^m long list, taking $\approx \mathrm{unif}\{1, \ldots, n\}$ time.
- External Heapsort is the first XSort to be considered for QuickXSort (Cantone and Cincotti 2002) and fulfills (X), with Π constant.

Other XSort

- For the third common MergeSort, Bottom-Up MergeSort, (X), specifically $\mathrm{Var}(M_n) = \mathrm{O}(n)$ is not true
  - For $n = 2^m + 1$, the last element is inserted into a sorted 2^m long list, taking $\approx \mathrm{unif}\{1, \ldots, n\}$ time.
- External Heapsort is the first XSort to be considered for QuickXSort (Cantone and Cincotti 2002) and fulfills (X), with Π constant.

Main result and Corollaries

Theorem (I.,Lesny, Neininger (2026))

For the number X_n of key comparisons required by QuickMergesort (top-down or queue), we have with Υ being a random 1-periodic continuous process that

$$\ell_2\left(\frac{X_n - n\log_2 n}{n}, \Upsilon(\{\log_2 n\})\right) \to 0 \quad (n \to \infty). \tag{1}$$

ℓ_2 convergence implies convergence in distribution and that, as $n \to \infty$,

$$\mathbb{E}[X_n] = n\log_2(n) + \Lambda_E(\log_2 n)n + o(n),$$
$$\mathrm{Var}(X_n) = \Lambda_V(\log_2 n)n^2 + o(n^2),$$

with continuous, 1-periodic functions $\Lambda_E(x) := \mathbb{E}[\Upsilon(x)], \Lambda_V(x) := \mathrm{Var}(\Upsilon(x))$ for $x \in [0,1]$.

Theorem (I.,Lesny, Neininger (2026))

For the number X_n of key comparisons required by QuickMergesort (top-down or queue), we have with Υ being a random 1-periodic continuous process that

$$\ell_2\left(\frac{X_n - n\log_2 n}{n}, \Upsilon(\{\log_2 n\})\right) \to 0 \quad (n \to \infty). \tag{1}$$

ℓ_2 convergence implies convergence in distribution and that, as $n \to \infty$,

$$\begin{aligned}\mathbb{E}[X_n] &= n\log_2(n) + \Lambda_E(\log_2 n)n + o(n),\\ \mathrm{Var}(X_n) &= \Lambda_V(\log_2 n)n^2 + o(n^2),\end{aligned}$$

with continuous, 1-periodic functions $\Lambda_E(x) := \mathbb{E}[\Upsilon(x)], \Lambda_V(x) := \mathrm{Var}(\Upsilon(x))$ for $x \in [0,1]$.

The buffer size

- How big is the buffer?
- For Mergesort, two lists must be merged in every step.
- With the normal implementation, the smaller of these is moved into the buffer, thus requiring $\left\lfloor \frac{n}{2} \right\rfloor$ of buffer space
- This can be improved to $\frac{n}{3}$ (Ping-Pong merge) and even further
- Remember that we assume that `XSort` needs a buffer of size $\sim \alpha n$ for some $\alpha \leq 1$.
- Normal MergeSort has $\alpha = \frac{1}{2}$, with Ping-Pong merge it has $\alpha = \frac{1}{3}$.

The buffer size

- How big is the buffer?
- For Mergesort, two lists must be merged in every step.
- With the normal implementation, the smaller of these is moved into the buffer, thus requiring $\left\lfloor \frac{n}{2} \right\rfloor$ of buffer space
- This can be improved to $\frac{n}{3}$ (Ping-Pong merge) and even further
- Remember that we assume that `XSort` needs a buffer of size $\sim \alpha n$ for some $\alpha \leq 1$.
- Normal MergeSort has $\alpha = \frac{1}{2}$, with Ping-Pong merge it has $\alpha = \frac{1}{3}$.

The buffer size

- How big is the buffer?
- For Mergesort, two lists must be merged in every step.
- With the normal implementation, the smaller of these is moved into the buffer, thus requiring $\left\lfloor \frac{n}{2} \right\rfloor$ of buffer space
- This can be improved to $\frac{n}{3}$ (Ping-Pong merge) and even further
- Remember that we assume that `XSort` needs a buffer of size $\sim \alpha n$ for some $\alpha \leq 1$.
- Normal MergeSort has $\alpha = \frac{1}{2}$, with Ping-Pong merge it has $\alpha = \frac{1}{3}$.

The buffer size

- How big is the buffer?
- For Mergesort, two lists must be merged in every step.
- With the normal implementation, the smaller of these is moved into the buffer, thus requiring $\left\lfloor \frac{n}{2} \right\rfloor$ of buffer space
- This can be improved to $\frac{n}{3}$ (Ping-Pong merge) and even further
- Remember that we assume that `XSort` needs a buffer of size $\sim \alpha n$ for some $\alpha \leq 1$.
- Normal MergeSort has $\alpha = \frac{1}{2}$, with Ping-Pong merge it has $\alpha = \frac{1}{3}$.

The buffer size

- `XSort` is more efficient than `QuickXSort`, so we use `XSort` to sort the bigger list if the smaller list is large enough to serve as buffer
- E.g. Mergesort has $\alpha = \frac{1}{2}$
  - if the pivot split into $\frac{1}{3}$ and $\frac{2}{3}$, MergeSort can run on the $\frac{2}{3}$ of the list using the $\frac{1}{3}$ as buffer.
  - If we split $\frac{1}{4}$ to $\frac{3}{4}$, then MergeSort can only run on the $\frac{1}{4}$.
- Then, together with (P'),

$$\frac{\#\text{ elements in next step of QuickXSort}}{n} \to Z \tag{P}$$

  in distribution
- This Z is

$$Z = \begin{cases} Z' & \text{if } \frac{\alpha}{1+\alpha} < Z' \leq \frac{1}{2} \text{ or } \frac{1}{1+\alpha} < Z' \leq 1 \\ 1 - Z' & \text{else.} \end{cases}$$

The buffer size

- `XSort` is more efficient than `QuickXSort`, so we use `XSort` to sort the bigger list if the smaller list is large enough to serve as buffer
- E.g. Mergesort has $\alpha = \frac{1}{2}$
  - if the pivot split into $\frac{1}{3}$ and $\frac{2}{3}$, MergeSort can run on the $\frac{2}{3}$ of the list using the $\frac{1}{3}$ as buffer.
  - If we split $\frac{1}{4}$ to $\frac{3}{4}$, then MergeSort can only run on the $\frac{1}{4}$.
- Then, together with (P'),

$$\frac{\#\text{ elements in next step of QuickXSort}}{n} \to Z \tag{P}$$

  in distribution
- This Z is

$$Z = \begin{cases} Z' & \text{if } \frac{\alpha}{1+\alpha} < Z' \le \frac{1}{2} \text{ or } \frac{1}{1+\alpha} < Z' \le 1 \\ 1 - Z' & \text{else.} \end{cases}$$

The buffer size

- `XSort` is more efficient than `QuickXSort`, so we use `XSort` to sort the bigger list if the smaller list is large enough to serve as buffer
- E.g. Mergesort has $\alpha = \frac{1}{2}$
  - if the pivot split into $\frac{1}{3}$ and $\frac{2}{3}$, MergeSort can run on the $\frac{2}{3}$ of the list using the $\frac{1}{3}$ as buffer.
  - If we split $\frac{1}{4}$ to $\frac{3}{4}$, then MergeSort can only run on the $\frac{1}{4}$.
- Then, together with (P'),

$$\frac{\#\text{ elements in next step of QuickXSort}}{n} \to Z \tag{P}$$

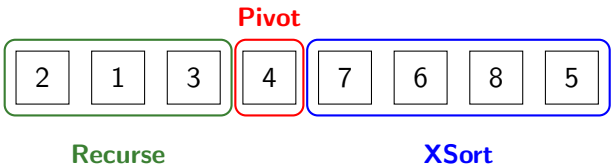
  in distribution
- This Z is

$$Z = \begin{cases} Z' & \text{if } \frac{\alpha}{1+\alpha} < Z' \leq \frac{1}{2} \text{ or } \frac{1}{1+\alpha} < Z' \leq 1 \\ 1 - Z' & \text{else.} \end{cases}$$

Contraction Method

The Recursion

Remember the iteration step:



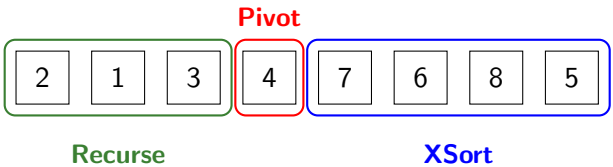
If I is the number of elements the recursion runs on,

$$X_n = X_I + n + o(n) + M_{n-1-I}$$

This gives a recursion for the rescaled process $(X_n - n \log_2 n)/n$

The Recursion

Remember the iteration step:



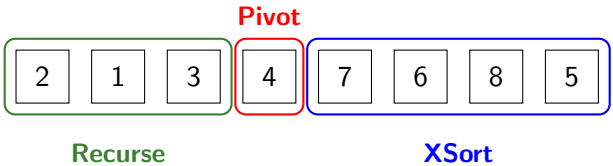
If I is the number of elements the recursion runs on,

$$X_n = X_I + n + o(n) + M_{n-1-I}$$

This gives a recursion for the rescaled process $(X_n - n \log_2 n)/n$

The Recursion

Remember the iteration step:



If I is the number of elements the recursion runs on,

$$X_n = X_I + n + o(n) + M_{n-1-I}$$

This gives a recursion for the rescaled process $(X_n - n \log_2 n)/n$

The Recursion limit

- By (P'), $I/n \xrightarrow{d} Z$
- The $n \log_2 n$ terms and n give an entropy-like term

$$\mathcal{E}(Z) := Z \log_2 Z + (1-Z)\log_2(1-Z) + 1 \geq 0$$

- This is 1 minus the entropy of a Bernoulli with prob. Z
- $\Rightarrow$ Minimal and 0 at $\frac{1}{2}$, 1 at 0 and 1
- The limit of the recursion is

$$Y_n \stackrel{d}{=} ZY_I + \mathcal{E}(Z) + (1-Z)\Pi(\log_2((1-Z)n))$$

- This is a contraction, so the contraction method can be used to give existence and convergence

The Recursion limit

- By (P'), $I/n \xrightarrow{d} Z$
- The $n \log_2 n$ terms and n give an entropy-like term

$$\mathcal{E}(Z) := Z \log_2 Z + (1-Z)\log_2(1-Z) + 1 \geq 0$$

- This is 1 minus the entropy of a Bernoulli with prob. Z
- $\Rightarrow$ Minimal and 0 at $\frac{1}{2}$, 1 at 0 and 1
- The limit of the recursion is

$$Y_n \stackrel{d}{=} ZY_I + \mathcal{E}(Z) + (1-Z)\Pi(\log_2((1-Z)n))$$

- This is a contraction, so the contraction method can be used to give existence and convergence

The Recursion limit

- By (P'), $I/n \xrightarrow{d} Z$
- The $n \log_2 n$ terms and n give an entropy-like term

$$\mathcal{E}(Z) := Z \log_2 Z + (1-Z)\log_2(1-Z) + 1 \geq 0$$

- This is 1 minus the entropy of a Bernoulli with prob. Z
- $\Rightarrow$ Minimal and 0 at $\frac{1}{2}$, 1 at 0 and 1
- The limit of the recursion is

$$Y_n \stackrel{d}{=} ZY_I + \mathcal{E}(Z) + (1-Z)\Pi(\log_2((1-Z)n))$$

- This is a contraction, so the contraction method can be used to give existence and convergence

The Recursion limit

- By (P'), $I/n \xrightarrow{d} Z$
- The $n \log_2 n$ terms and n give an entropy-like term

$$\mathcal{E}(Z) := Z \log_2 Z + (1 - Z) \log_2(1 - Z) + 1 \geq 0$$

- This is 1 minus the entropy of a Bernoulli with prob. Z
- $\Rightarrow$ Minimal and 0 at $\frac{1}{2}$, 1 at 0 and 1
- The limit of the recursion is

$$Y_n \stackrel{d}{=} ZY_I + \mathcal{E}(Z) + (1 - Z)\Pi(\log_2((1 - Z)n))$$

- This is a contraction, so the contraction method can be used to give existence and convergence

The Recursion limit

- By (P'), $I/n \xrightarrow{d} Z$
- The $n \log_2 n$ terms and n give an entropy-like term

$$\mathcal{E}(Z) := Z \log_2 Z + (1 - Z) \log_2(1 - Z) + 1 \geq 0$$

- This is 1 minus the entropy of a Bernoulli with prob. Z
- $\Rightarrow$ Minimal and 0 at $\frac{1}{2}$, 1 at 0 and 1
- The limit of the recursion is

$$Y_n \stackrel{d}{=} ZY_I + \mathcal{E}(Z) + (1 - Z)\Pi(\log_2((1 - Z)n))$$

- This is a contraction, so the contraction method can be used to give existence and convergence

The periodic limit

In

$$Y_n \stackrel{d}{=} ZY_I + \mathcal{E}(Z) + (1-Z)\Pi(\log_2((1-Z)n))$$

the limit is still dependant on n, but one can complete it to a full periodic function Υ

$$\Upsilon(x) \stackrel{d}{=} Z\Upsilon + \mathcal{E}(Z) + (1-Z)\Pi(x + \log_2(1-Z))$$

Properties of the Limit

The expectation

From

$$\Upsilon(x) \stackrel{d}{=} Z\Upsilon + \mathcal{E}(Z) + (1-Z)\Pi(x + \log_2(1-Z))$$

we see

$$\int_0^1 \mathbb{E}\Upsilon(x)\mathrm{d}x = \frac{\mathbb{E}[\mathcal{E}(Z)]}{1-\mathbb{E}Z} + \int_0^1 \Pi(x)\mathrm{d}x$$

$$\max(\mathbb{E}\Upsilon) \le \frac{\mathbb{E}[\mathcal{E}(Z)]}{1-\mathbb{E}Z} + \max(\Pi)$$

$$\min(\mathbb{E}\Upsilon) \ge \frac{\mathbb{E}[\mathcal{E}(Z)]}{1-\mathbb{E}Z} + \min(\Pi)$$

So we want Z small, but bigger than $\frac{\alpha}{1+\alpha}$ and close to $\frac{1}{2}$ (so that $\mathcal{E}(Z)$ small), hence we use multiple pivots.

The expectation

From

$$\Upsilon(x) \stackrel{d}{=} Z\Upsilon + \mathcal{E}(Z) + (1-Z)\Pi(x + \log_2(1-Z))$$

we see

$$\int_0^1 \mathbb{E}\Upsilon(x)\mathrm{d}x = \frac{\mathbb{E}[\mathcal{E}(Z)]}{1-\mathbb{E}Z} + \int_0^1 \Pi(x)\mathrm{d}x$$

$$\max(\mathbb{E}\Upsilon) \leq \frac{\mathbb{E}[\mathcal{E}(Z)]}{1-\mathbb{E}Z} + \max(\Pi)$$

$$\min(\mathbb{E}\Upsilon) \geq \frac{\mathbb{E}[\mathcal{E}(Z)]}{1-\mathbb{E}Z} + \min(\Pi)$$

So we want Z small, but bigger than $\frac{\alpha}{1+\alpha}$ and close to $\frac{1}{2}$ (so that $\mathcal{E}(Z)$ small), hence we use multiple pivots.

Expectation

Normal Top-down MergeSort has

$$\mathbb{E}[M_n] = n \log_2 n - (1.25265 \pm 0.01185)n + o(n)$$

Corollary (Edelkamp, Weiß, and Wild (2020))

For the X_n from Theorem 1 using Top-down Mergesort, we have

$$\mathbb{E}[X_n] = n \log_2 n - (0.3407 \pm 0.01185)n + o(n) \quad \text{for median-of-1,}$$
$$\mathbb{E}[X_n] = n \log_2 n - (0.8477 \pm 0.01185)n + o(n) \quad \text{for median-of-3.}$$

For Queue-Mergesort,

$$\mathbb{E}[X_n] = n \log_2 n - (0.0450 \pm 0.04303)n + o(n) \quad \text{for median-of-1,}$$
$$\mathbb{E}[X_n] = n \log_2 n - (0.5520 \pm 0.04303)n + o(n) \quad \text{for median-of-3.}$$

Expectation

Normal Top-down MergeSort has

$$\mathbb{E}[M_n] = n \log_2 n - (1.25265 \pm 0.01185)n + o(n)$$

Corollary (Edelkamp, Weiß, and Wild (2020))

For the X_n from Theorem 1 using Top-down Mergesort, we have

$$\mathbb{E}[X_n] = n \log_2 n - (0.3407 \pm 0.01185)n + o(n) \quad \textit{for median-of-1,}$$
$$\mathbb{E}[X_n] = n \log_2 n - (0.8477 \pm 0.01185)n + o(n) \quad \textit{for median-of-3.}$$

For Queue-Mergesort,

$$\mathbb{E}[X_n] = n \log_2 n - (0.0450 \pm 0.04303)n + o(n) \quad \textit{for median-of-1,}$$
$$\mathbb{E}[X_n] = n \log_2 n - (0.5520 \pm 0.04303)n + o(n) \quad \textit{for median-of-3.}$$

Expectation

Normal Top-down MergeSort has

$$\mathbb{E}[M_n] = n \log_2 n - (1.25265 \pm 0.01185)n + o(n)$$

Corollary (Edelkamp, Weiß, and Wild (2020))

For the X_n from Theorem 1 using Top-down Mergesort, we have

$$\mathbb{E}[X_n] = n \log_2 n - (0.3407 \pm 0.01185)n + o(n) \quad \textit{for median-of-1,}$$
$$\mathbb{E}[X_n] = n \log_2 n - (0.8477 \pm 0.01185)n + o(n) \quad \textit{for median-of-3.}$$

For Queue-Mergesort,

$$\mathbb{E}[X_n] = n \log_2 n - (0.0450 \pm 0.04303)n + o(n) \quad \textit{for median-of-1,}$$
$$\mathbb{E}[X_n] = n \log_2 n - (0.5520 \pm 0.04303)n + o(n) \quad \textit{for median-of-3.}$$

Variance

- Because the variance for `XSort` is assumed to be $\mathrm{O}(n)$, the variance is dominated by the pivot selection
- Additional variance from evaluating Π at a random place $x + \log_2(1 - Z)$
- If Π was constant,

$$\mathrm{Var}(\Upsilon) = \frac{\mathrm{Var}\left(\frac{\mathbb{E}[\mathcal{E}(Z)]}{1-\mathbb{E}[Z]} Z + \mathcal{E}(Z)\right)}{1 - \mathbb{E}[Z^2]}.$$

- For median-of-3 pivot ($h = 2$) and $\alpha = \frac{1}{2}$, this is roughly 0.1068.

Variance

- Because the variance for `XSort` is assumed to be $\mathrm{O}(n)$, the variance is dominated by the pivot selection
- Additional variance from evaluating Π at a random place $x + \log_2(1 - Z)$
- If Π was constant,

$$\mathrm{Var}(\Upsilon) = \frac{\mathrm{Var}\left(\frac{\mathbb{E}[\mathcal{E}(Z)]}{1-\mathbb{E}[Z]} Z + \mathcal{E}(Z)\right)}{1 - \mathbb{E}[Z^2]}.$$

- For median-of-3 pivot ($h = 2$) and $\alpha = \frac{1}{2}$, this is roughly 0.1068.

Fourier coefficients

- Υ is a periodic function, so study its Fourier coefficients
- The k-th Fourier coefficient $c_{\Upsilon,k}$ of Υ fulfils the recursion

$$c_{\Upsilon,k} \stackrel{d}{=} Z^{1+\chi_k} c_{\Upsilon,k} + \mathcal{E}(Z)\mathbf{1}(k=0) + (1-Z)^{1+\chi_k} c_{\Pi,k}.$$

- From this, explicit forms for the Fourier coefficients of mean and variance can be obtained
- The Fourier coefficients (except the 0-th) of the mean are smaller or equal than those of Π, implying the $\mathbb{E}[\Upsilon]$ is smoother than Π.

Fourier coefficients

- Υ is a periodic function, so study its Fourier coefficients
- The k-th Fourier coefficient $c_{\Upsilon,k}$ of Υ fulfils the recursion

$$c_{\Upsilon,k} \stackrel{d}{=} Z^{1+\chi_k} c_{\Upsilon,k} + \mathcal{E}(Z)\mathbf{1}(k=0) + (1-Z)^{1+\chi_k} c_{\Pi,k}.$$

- From this, explicit forms for the Fourier coefficients of mean and variance can be obtained
- The Fourier coefficients (except the 0-th) of the mean are smaller or equal than those of Π, implying the $\mathbb{E}[\Upsilon]$ is smoother than Π.

Conclusion

- In QuickMergeSort, you pay a small expected $\approx 0.4n$ of speed to not require extra buffer space $+ \approx 0.3n$ std deviation
- We identified mean, variance and limit distribution of the number of comparisons in QuickMergeSort
- Mean and variance are both oscillate with $\log_2 n$
- Open Question: If we take as pivot the median of a sample whose size increases in n, $Z \to \frac{1}{2}$ and $\Upsilon = \Pi$ has no variance. What is then the limit distribution?

Conclusion

- In QuickMergeSort, you pay a small expected $\approx 0.4n$ of speed to not require extra buffer space $+ \approx 0.3n$ std deviation
- We identified mean, variance and limit distribution of the number of comparisons in QuickMergeSort
- Mean and variance are both oscillate with $\log_2 n$
- Open Question: If we take as pivot the median of a sample whose size increases in n, $Z \to \frac{1}{2}$ and $\Upsilon = \Pi$ has no variance. What is then the limit distribution?

Thanks